

从零正着推出 Tarjan 强连通分量算法

low-link 的发现式推导 · 三类边与 dfn/low 之争的完整拉扯 · 正确性证明 · 逐个 bug 复盘 · 图灵奖与第一性原理

技术手记 · jsjhoubo.github.io

目录

- 0 · 引子:背一个算法,和拥有一个算法
- 1 · 把问题钉死:有向图、可达性、等价关系、SCC
- 2 · 最笨的解法 $O(V^4)$,以及它替我们指出的方向
- 3 · 逼出 low:一个不是被定义、而是被"需要"出来的量
- 4 · 三类边的拉扯(本文核心一):哪条信息能用,哪条不能
- 5 · dfn[j] 还是 low[j]:那个最微妙的岔路
- 6 · 根判据与栈:如何一次性把一个分量捞干净
- 7 · 正确性证明:把前面所有判断串成一条不漏的链
- 8 · 从证明到代码的那道沟(本文核心二):逐个 bug 复盘
- 9 · 全部代码 + 七个测试用例
- 10 · 它从哪来:图灵奖与 1972 那篇论文
- 11 · 余论:欧拉七桥、建模 > 求解、第一性原理与手稿

0 · 引子:背一个算法,和拥有一个算法

几乎所有讲 Tarjan 的材料都长一个样:先甩出 dfn 和 low 的定义,再告诉你"遇到三类边分别这样更新 low",最后贴代码。你照着背,能 AC,但只要面试官换个角度追问——"为什么这条边用 dfn[j] 而那条用 low[j]?"——"为什么 low 初值要这么设?"——背下来的那层就塌了,因为你手里只有结论,没有把结论生出来的那条推理链。

本文要做的是相反的事:不预设 low 是什么,而是从一个最朴素的问题出发,把 low、把闭合判据、把那个栈,一步步"需要"出来。更重要的是,我会把推导过程中真实的弯路、错判、自我反驳全部保留下来——因为那些"我本以为 X,结果不对,因为 Y,所以改成 Z"的拉扯,才是真正让你"拥有"这个算法、而不只是"记住"它的东西。

这篇文章里有大量"疑 / 答 / 修"的对话块。它们不是排版花样,而是刻意保留的**思维现场**:疑=我卡住或起疑的地方,答=当时的推进,修=后来发现前面错了、怎么纠回来。读的时候请跟着每一个"疑"先自己想一想再往下看——这才是"正着推"的正确打开方式。

1 · 把问题钉死:有向图、可达性、等价关系、SCC

任何推导都得先有一块不动的地基。我们要解的对象——"强连通分量"——必须先有一个不依赖任何算法的、纯粹基于定义的描述。后面所有的"为什么这样做对",最终都要能追溯回这一节。

1.1 可达性

有向图 $G = (V, E)$, V 是顶点集, $E \subseteq V \times V$ 是有向边集。称 u **可达** v (记 $u \rightsquigarrow v$), 当且仅当存在一条有向路径 $u = x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_k = v$, 其中 $k \geq 0$ 且每条 $(x_i, x_{i+1}) \in E$ 。特别地, $k=0$ 时 $u \rightsquigarrow u$ 恒成立——**每个点零步可达自身**。这个"零步"看似废话, 后面会发现它正是"单点也是合法 SCC"的根。

1.2 相互可达, 以及它是个等价关系

在 V 上定义二元关系: $u \sim v \iff (u \rightsquigarrow v) \text{ 且 } (v \rightsquigarrow u)$, 即两点能互相走到。我们逐条验证它是**等价关系**, 因为这决定了"SCC 是干净的划分"这件事:

- **自反**: $u \sim u$, 由零步可达立即得到。
- **对称**: 定义里 $(u \rightsquigarrow v)$ 与 $(v \rightsquigarrow u)$ 地位对称, 交换 u, v 不变。
- **传递**: 若 $u \sim v$ 且 $v \sim w$, 则 $u \rightsquigarrow v \rightsquigarrow w$ 故 $u \rightsquigarrow w$; 反方向 $w \rightsquigarrow v \rightsquigarrow u$ 故 $w \rightsquigarrow u$ 。于是 $u \sim w$ 。

1.3 强连通分量 = 等价类

既然 $\sim$ 是等价关系, 它就把 V 切成若干**互不相交、并起来是全集**的等价类。每个等价类就是一个**强连通分量 (SCC)**: $[u] = \{v \in V \mid u \sim v\}$ 。等价的说法: SCC 是 V 的一个**极大(maximal)**子集 C , 使 C 内任意两点互相可达——"极大"意味着**再往里加任何一个点, 都会破坏"两两互相可达"这个性质**。

"极大"这个词后面会引发一次真实的争论。有人会问: 四个点互相可达的集合, 取它三个点的子集, 这三点在原图里不也两两互相可达吗? 是的——但那个三点集叫**强连通子图**, 不叫**强连通分量**; component 这个词**本身含 maximal**。强连通子图可以嵌套, 强连通分量(等价类)不能。术语的这一刀, 后面证明"分量互不嵌套"时要用到。

1.4 一个立刻能榨出来的推论: 缩点后是 DAG

既然 $\sim$ 把 V 划成不相交的等价类, 那么**任意两个不同的 SCC 之间, 可达性必定是单向的**: 不可能既有 $C_1 \rightsquigarrow C_2$ 又有 $C_2 \rightsquigarrow C_1$ ——否则 C_1, C_2 里的点就两两互相可达, 该并成同一个等价类, 与"它们是两个不同分量"矛盾。把每个 SCC 缩成一个点, 分量间的边就构成一张**有向无环图(DAG)**。这个推论我们暂时用不上, 但它是"为什么强连通分量值得单独拎出来"的根本理由: 它让任意有向图都能被压成一张 DAG。

2 · 最笨的解法 $O(V^4)$, 以及它替我们指出的方向

不要一上来就追求聪明。先写出最直白、一定正确、但很慢的解法——它往往会**亲口告诉你瓶颈在哪**, 而瓶颈正是优化的入口。

2.1 直接照定义做

要判断 u, v 是否同一个 SCC, 就照 1.2 的定义来: 从 u 做一遍 DFS/BFS 看能否到 v , 再从 v 做一遍看能否回到 u ; 两边都能, 则 $u \sim v$ 。单次遍历是 $O(V+E)$ 。对所有 $C(V, 2) \approx V^2$ 个点对各判一次, 把判定为相互可达的点用并查集 union 起来, 最后每个连通块就是一个 SCC。

总复杂度: V^2 个点对 $\times$ 每对 $O(V+E) = O(V^2(V+E))$, 稠密图下约 $O(V^4)$ 。并查集本身近 $O(V \alpha(V))$, 可忽略; 瓶颈完全在那 V^2 遍遍历。

2.2 它浪费在哪——这才是重点

这个解法慢得离谱,但它**暴露了浪费的确切位置**:你从 u 出发做的那一遍 DFS,其实**顺手就知道了 u 能到达的所有点**——这是一整片信息。可你只用它回答了" u 能不能到某个特定的 v "这一个是非题,然后把整片信息扔了。下一对点你又从头再来。同一段可达性,被重复计算了 $O(V)$ 量级的次数。

钥匙就在这句话里:一遍 DFS 蕴含的信息远多于" u 能否到 v "。如果我们能设计一个量,让**一遍 DFS 在回溯的过程中顺手就把"谁和谁同属一个 SCC"算出来**,就能把 V^2 遍压成**常数遍线性 DFS**,做到 $O(V+E)$ 。——这正是 Tarjan 的目标,也是 low 这个量诞生的**动机**。注意:low 不是天上掉下来的定义,它是"我想在一遍 DFS 里顺手定出 SCC"这个需求逼出来的产物。

3 · 逼出 low:一个不是被定义、而是被"需要"出来的量

现在开始真正的推导。我们不写"low 的定义是……",而是问:**要在一遍 DFS 里判断分量边界,我到底需要知道什么?**

3.1 DFS 树,和那个核心问题

DFS 跑在有向图上,会沿着"首次访问"的边形成一棵 DFS 树(其余边是回边/前向边/横叉边)。当 DFS 从某个点 u **回溯**(即 u 的所有出边都处理完、准备返回)时,我想就地判断一件事:

以 u 为根的这棵 DFS 子树,是不是恰好**"封顶"了一个强连通分量**?
——等价地问: u 的子树里,有没有任何一个点,能**"绕回到 u ,或绕回到 u 之上(更早被访问的祖先)"**?

疑 ▶ 为什么"能不能绕回 u 之上"这个问题,就等价于" u 是不是分量的顶"?

答 ▶ 因为强连通要求"互相可达"。 u 的子树里的点,顺着树边天然能被 u 到达(u 在它们上面);反方向能不能回到 u 或 u 的祖先,决定了它们和 u (乃至 u 的祖先)能不能**互相可达**、从而同属一个 SCC。

- 若子树里**没有任何点**能绕回 u 之上 → 这坨点"出不去",回不到更高的祖先,那么 u 就是这一团的最高点, u 封顶,分量到 u 为止。
- 若子树里**有点**能绕回 u 的某个祖先 a → 那么 u 、 a 和这条绕回的路径串成了一个更大的环,环上的点互相可达, u 当不了顶,真正的"顶"在 a 那一层甚至更高。

于是,我需要的核心信息浮出水面了:从 u 的子树出发,**最远能"往回够"到多早的那个点是谁?**只要我能为每个 u 维护这个"最远能够回到的点",就能用它和 u 自身的位置一比,判断 u 是不是封顶。**把这个" u 及其子树最远能往回够到的点"记作 $low[u]$** 。——看清楚:low 是被"判断封顶"这个需求逼出来的,不是我先定义它再去用。

3.2 第一个精度问题:记"节点"还是记"序号"?

疑 ▶ $low[u]$ 要存"那个最早的点"——那存什么?存节点本身(它的 id)吗?

答► 不必。我们只关心"谁更早",这是一个**比较**操作,不需要节点的身份,只需要一个能比大小的标量。所以给每个点在**首次被访问**时盖一个时间戳:递增的**发现序号** $\text{dfn}[u]$ (它是第几个被访问到的)。 $\text{low}[u]$ 就存这种序号——"最远能往回够到多早"直接退化成"最小的 dfn "。节点 id 完全不进 low (它只在最后捞分量成员时、待在栈里有用)。

3.3 第二个精度问题:是"发现序"还是"深度"?——一个容易踩的坑

疑► 那个序号,用"DFS 深度"(离根几层)行不行?直觉上越浅的越像祖先啊。

修► 不行,这里有个真实的坑。**深度(depth)和发现序(discovery order)不是一回事**。反例:根有左右两棵子树,DFS 先把**左子树整棵走完**再去右子树。那么左子树里一个**很深**的点,它的发现序号可能**小于**右子树里一个**很浅**的点——因为它先被访问到。我们要表达的是"谁先被发现"(全局时间序),而不是"谁层数浅"(局部结构)。**用 dfn (发现序)才对,用 depth (深度)会错**。这一条之所以重要,是因为 low 的整条逻辑都建立在"沿时间序往回够"上,一旦把时间序换成深度,后面"栈中 dfn 递增"这个性质(第 6 节)就不成立了,整个收口机制会崩。

3.4 把 low 的语义形式化:一个精确定义与三条不变量

前面 low 是"逼"出来的直觉,现在把它写成一个不留歧义的形式化定义,后续证明全部以此为准。设 DFS 进行到某时刻,定义:

$\text{low}[u] = \min \{ \text{dfn}[w] \mid w \text{ 可由 } u \text{ 出发,沿 } u \text{ 子树内的若干树边、再加至多一条指向"当前未闭合顶点"的非树边到达} \}$ 。

这个"树边若干 + 至多一条未闭合非树边"的刻画,正是第 4、5 节所有判据的来源:树边随便走(子结构),但跨出去的那一步只能落在"还活着(未闭合)"的点上,且只需一步(因为更远的早点会经由别的树边链自然汇聚)。算法维护的三条**不变量**(invariant)是:

- **I1(底)**:任意时刻 $\text{low}[u] \leq \text{dfn}[u]$ (进门即设 $\text{low}[u] = \text{dfn}[u]$,之后只减不增)。**"至少能回到自己"**。
- **I2(栈即未闭合)**:顶点 u 在栈中 $\iff u$ 已被访问但其所属 SCC 尚未收口。这条把"闭合状态"和"在不在栈里"绑定,是第 6 节双重作用的根基。
- **I3(栈序)**:栈中元素从底到顶 dfn 严格递增;任何时刻栈内顶点构成若干条"从某祖先到当前 DFS 路径"的链。

这三条不变量会在第 7 节的证明里被反复引用。把它们提前钉死的好处是:后面每一步操作,我们都能问"它是否保持了 I1-I3",从而把"算法对不对"化为"不变量有没有被破坏"——这是把直觉推导升级成严格证明的标准手法。

4 · 三类边的拉扯(本文核心一):哪条信息能用,哪条不能

这一节是整个 Tarjan 最容易写错、也最值得慢慢拉扯的地方。我们已经知道 low 要"沿子树向上汇聚取 $\min$ ",但具体地——处理 u 的每一条出边 ($u \rightarrow j$) 时,到底拿 j 的什么信息来更新 $\text{low}[u]$?我会把真实的试错过程一步步铺开。

4.1 环如何形成,以及"合并"的图景

先把"环"看清楚。沿树路径 根 $\rightarrow \dots \rightarrow a$, 若 a 有一条边指回路径上更早的祖先, 就闭成一个环; 环上任意两点互相可达, 所以整条环同属一个等价类。更进一步——**环能经由共享的点串联合并**: a 闭出一个环, 而这个环上的另一个点又有边够到了更早的祖先, 两个环共享了点, 于是合并成一个更大的强连通块, 真正的根被一路顶到那个最早的祖先。这正是 low 必须**沿子树向上传播、并取 $\min$** 的原因: 子结构能够到多早, 要汇报给父亲, 父亲再汇报给祖父……直到某个点发现"我子树最远也只能够到我自己", 它就是根。

4.2 第一直觉, 以及它的反例

疑 ▶ 那简单啊: 处理每条边 $u \rightarrow j$, 直接 $\text{low}[u] = \min(\text{low}[u], \text{low}[j])$ 不就行了? j 能够到多早, u 经由 j 也能够到多早。

答 ▶ 对"未访问的树边孩子"成立——递归处理完 j , $\text{low}[j]$ 已经收敛, 拿来更新 u 没问题。但对"已经访问过的 j "呢? 你默认了" j 能够到的早点, u 经由这条边也能够到"。这句话**不总成立**。

疑 ▶ 为什么不总成立? j 不就在那儿吗, $\text{low}[j]$ 不就是它能回到的最早点吗?

修 ▶ 反例: 设想 j 属于另一棵**已经整个走完、已经被认定为一个独立 SCC** 的子树。这种 j , 它的 $\text{low}[j]$ 是个有效的旧值没错, 但它所在的分量**已经收口、与 u 之间没有"活的"回路了**。这时若把 $\text{low}[j]$ 拉进 $\text{low}[u]$, 等于把一个**跟 u 根本连不成环**的早点, 错误地算作" u 能够回到的点"—— u 的 low 被污染, 本不该并的两个分量会被错并。

所以"无论 j 访问与否都用 $\text{low}[j]$ "是错的。关键不在" j 访问没访问", 而在" j 所属的分量, 闭合了没有"。

4.3 一个关键的中间发现: $\text{low}[j]$ 指向的, 未必是祖先, 也可能是"兄弟"

疑 ▶ 等等, 我一开始以为 $\text{low}[j]$ 指向的总是 j (或 u) 的祖先。是这样吗?

答 ▶ 不是——这是个值得停下来发现。经由非树边, $\text{low}[j]$ 可以指向一个并非直系祖先的点, 比如另一棵子树里的"兄弟"节点(横叉边指过去的)。这就把"已访问的 j "劈成了两种本质不同的情况:

- 那个兄弟所在的子树**还在活动**(尚未收口) $\rightarrow u$ 还可能经由它绕回某个共同祖先(LCA 或更高)构成环 $\rightarrow \text{low}[j]$ 有效;
- 那个兄弟所在的子树**已经封闭** $\rightarrow$ 它和 u 之间已无活路, 环连不起来 $\rightarrow \text{low}[j]$ 无效。

"祖先 vs 兄弟"这个区分, 正是"已访问的 j 必须再分两类"的根源。

4.3b 把"合并"走一遍: 两环共享一点的具体 trace

光说"环经由共享点合并"还是抽象, 我们拿一个最小的共享点图走一遍, 看 low 怎么把根一路顶到共享点。图: $0 \rightarrow 1, 1 \rightarrow 2, 2 \rightarrow 0$ (环 A) 与 $0 \rightarrow 3, 3 \rightarrow 4, 4 \rightarrow 0$ (环 B), 两环共享顶点 0, 整图应是一个 SCC (0, 1, 2, 3, 4)。设 0 的出边顺序为 [1, 3]。

动作	关键变化	说明
$\text{dfs}(0) \rightarrow \text{dfs}(1) \rightarrow \text{dfs}(2)$	$\text{dfn}=0, 1, 2; \text{low}=0, 1, 2; \text{栈}[0, 1, 2]$	沿环 A 下探
$2 \rightarrow 0$ (回边)	$\text{low}[2] = \min(2, \text{dfn}/\text{low}[0]=0) = 0$	

		0 在栈、未闭合 → 可用,环 A 把 low[2] 拉到 0
回 1、回 0	$\text{low}[1] = \min(1, \text{low}[2]=0)=0; \text{low}[0] = \min(0, \text{low}[1])=0$	环 A 的信息汇聚到 0;但 0 的出边还没走完(还有 3) ,先不判根
0→3→dfs(4)	$\text{dfn}[3]=3, \text{dfn}[4]=4; \text{low}=3, 4; \text{栈}[0, 1, 2, 3, 4]$	沿环 B 下探
4→0(回边)	$\text{low}[4] = \min(4, \text{low}[0]=0)=0$	0 仍在栈、仍未闭合(关键!)→ 环 B 也把 low[4] 拉到 0
回 3、回 0	$\text{low}[3]=0; \text{low}[0] = \min(0, \text{low}[3]=0)=0$	环 B 的信息也汇聚到 0
dfs(0) 收尾	$\text{low}[0] == \text{dfn}[0] == 0 \rightarrow 0 \text{ 是根}$	弹栈 $\text{dfn} \geq 0$:弹出 4,3,2,1,0 → 一个 SCC (0, 1, 2, 3, 4)

合并的机制在这张表里看得清清楚楚:**两个环都靠"0 还在栈、还没收口"这一点,各自把 low 拉到 0**,于是 0 成了两环共同的根,一次弹栈把五个点全收进同一个分量。设想若环 A 先于环 B 把 0 收口了(它不会,因为 0 的出边没走完),环 B 的回边 4→0 就会被 processed 挡掉,(3, 4) 会被错误地单独成 SCC——这从反面说明:**根必须等自己所有出边都处理完才判 $\text{low} == \text{dfn}$,绝不能在循环中途收口**。这也呼应了第 8 节里"根判据写在循环内部"那个坑。

判据(本文推导出的核心结论):计算 $\text{low}[u]$ 时,对邻居 j ,只有当 j 所属的分量"尚未闭合"时,才执行 $\text{low}[u] = \min(\text{low}[u], \text{low}[j])$;已闭合的一律跳过。判据是" j 对应的点闭没闭合",和 $\text{vis}[j]$ 是否为 true 无关——因为"已访问"里面还分"闭合"与"未闭合"两种。

疑 ▶ 那我能不能省事,直接"只看 $\text{low}[j]$ 的数值"来判断 j 闭没闭合?比如 $\text{low}[j]$ 小到某个程度就算闭合?

修 ▶ 不能,这是最后一个要堵的洞。已闭合点的 $\text{low}[j]$,仍然是一个有效的小数值,和未闭合活动点的 $\text{low}[j]$ 长得一模一样。光看数值,你区分不了"还能回溯的活动点"和"已封闭的横叉点"。所以"闭合与否"是 j 的一个独立状态,必须额外记录一个标记,不能从 low 的数值反推。——这个"额外标记",到第 6 节会发现,正好就是" j 还在不在那个栈里"。

邻居 j 的身份	用什么更新 $\text{low}[u]$	理由
未访问(树边)	递归后用 $\text{low}[j]$	j 探索已完成, $\text{low}[j]$ 可信
已访问 · 分量未闭合	可用(见第 5 节: $\text{dfn}[j]$ 或 $\text{low}[j]$)	仍在活动路径,能与 u 成环
已访问 · 分量已闭合	跳过,不更新	与 u 无活路,强行并入会错并

4.5 那两类"看起来吓人"的边:前向边与横叉边,为什么自动被处理

DFS 在有向图上其实会产生四类边:树边、回边、**前向边**(指向自己子树里已访问的后代)、**横叉边**(指向另一棵子树、与自己无祖先后代关系的点)。前面只仔细谈了树边和回边。那前向边、横叉边会不会破坏 low?好消息是:它们都被前面的判据自动正确处理了,不需要任何特判。逐个看:

前向边 $u \rightarrow w$ (w 是 u 的后代,已访问)

w 在 u 的子树里,意味着 $\text{dfn}[w] > \text{dfn}[u]$,而且 w 这条树边的子结构早已通过**正常的树边递归**把它的 low 贡献给了 u 。这条前向边即便参与 $\min(\text{low}[u], \text{low}[w])$,贡献的也是一个 $\geq \text{dfn}[u]$ 的值(w 是后代,它能往回够到的最早点不会早于 u ,除非本就同环——而那种情况树边已经处理)。所以前向边要么无效(拉不小 $\text{low}[u]$),要么贡献的信息树边早已贡献过——它不会把 $\text{low}[u]$ 错误地拉小,也就不会破坏正确性。

横叉边 $u \rightarrow x$ (x 在另一棵子树)

这是真正需要判据保护的一类。在有向图里,横叉边总是指向**更早被访问完**的那棵子树(否则就不是横叉而是回边了),所以 x 此刻**几乎总是已经闭合**(它那棵子树先收口了)。于是 `!processed[low[x]]` 为假,这条横叉边被**直接跳过**——正是第 4.4 节判据的用武之地。万一 x 恰好还未闭合(罕见但可能,取决于 DFS 访问次序),那它就退化成"已访问、未闭合"的情形,按第 5 节用 $\text{dfn}[x]$ 安全更新即可。**两种子情况都被既有判据覆盖,无需为横叉边单写逻辑。**

这就是 Tarjan 优雅的另一面:你不需要在代码里区分"这条是回边还是横叉边还是前向边"——那是教科书分类法,实现时**根本不必判断边的类型**。你只需要一个统一动作:" j 未访问就递归,然后只要 j 未闭合就 $\min(\text{low}[u], \text{low}[j])$ "。四类边的正确性,全被"未闭合才用"这一条判据 + "low 进门设 dfn "这一条初值**自动兜住**。想清楚这点,代码反而比脑子里的边分类更简单。

5 • $\text{dfn}[j]$ 还是 $\text{low}[j]$:那个最微妙的岔路

第 4 节我们说,对"已访问、未闭合"的 j 可以用来更新 $\text{low}[u]$ 。但具体用 j 的哪个量—— **$\text{dfn}[j]$ 还是 $\text{low}[j]$** ?这是 Tarjan 实现里最微妙、最多人含糊带过的一处。我们把它彻底拉清楚。

疑 ▶ 对一条回边 $u \rightarrow j$ (j 是还在栈里的祖先),我该写 $\text{low}[u] = \min(\text{low}[u], \text{low}[j])$ 还是 $\text{low}[u] = \min(\text{low}[u], \text{dfn}[j])$?既然 $\text{low}[j]$ 信息更多(它可能比 $\text{dfn}[j]$ 还小),用 $\text{low}[j]$ 不是更好、能传得更远吗?

答 ▶ 先想清楚这条边**到底保证了什么**。边 $u \rightarrow j$ 只保证" u 能一步走到 j 这个点"。它**不保证**" u 能到达 j 子树更远处能到的那个点($\text{low}[j]$ 指向的地方)"。那个更远的点是 j 自己能到,不代表 u 经由**这一条边**就能到。

答 ▶ 再想:如果 u 和 j 本来就在同一个还在生长的大环里,那个"更早的点"的 dfn ,会经由别的路径(**树边链**)自然地汇聚到 u ,根本不需要这条回边去搬运 $\text{low}[j]$ 。所以——**对回边/指向活动点的非树边,用 $\text{dfn}[j]$ 就够,而且永远安全;用 $\text{low}[j]$ 只在 " j 确实还在栈里(未闭合)"时才安全。**

修► 为什么 $\text{dfn}[j]$ "永远安全"而 $\text{low}[j]$ "有条件"? 因为 $\text{dfn}[j]$ 一旦在 j 首访时定下,就永不改变;而 $\text{low}[j]$ 是动态的、可能还没收敛——一个还在递归中、探索没结束的 j ,它的 $\text{low}[j]$ 此刻可能只是个中间值,后面还会被拉小。用一个"还会变的中间值"去更新别人,是危险的。 $\text{dfn}[j]$ 是定死的锚点,所以标准实现普遍选 $\text{dfn}[j]$ 。

结论:① 树边孩子(递归已返回)→ 用 $\text{low}[j]$ (已收敛,可信);② 回边 / 指向活动点的非树边 → 用 $\text{dfn}[j]$ (定死的锚点,永远安全)。两种写法在" j 在栈里"时等价,但 $\text{dfn}[j]$ 的版本更不容易写错。

本文配套代码用的是一种等价的紧凑写法:统一 `if (!processed[low[j]])`

$\text{low}[r] = \min(\text{low}[r], \text{low}[j])$ 。它之所以也对,是因为(a)进入 DFS 时即令 $\text{low}[r] = \text{dfn}[r]$, (b)未闭合的 j 的 $\text{low}[j]$ 在"它和 r 同环"时会经由树边链正确汇聚, (c)已闭合的 j 被 `processed` 挡掉。理解了 dfn/low 之争,你才看得懂这种紧凑写法为何成立。

疑► 那"如果 $\text{low}[j] < \text{dfn}[j]$ 呢"——丢掉 $\text{low}[j]$ 不是亏了吗?

答► 不亏。 $\text{low}[j] < \text{dfn}[j]$ 意味着 j 还能往回够到比自己更早的点;但那个"更早的点"如果真和 u 同属一个环,它的 dfn 会通过环上的**树边路径**传到 u ,不靠这条回边。回边只需老老实实贡献"我够到了 j 这个点($\text{dfn}[j]$)"这一条确定无疑的信息即可。把不确定的 $\text{low}[j]$ 丢给上游,反而可能在 j 已闭合时造成错并。**保守(用 $\text{dfn}[j]$)在这里恰恰是正确,而非将就。**

5.1 反过来看:如果不挡"已闭合",错并会怎样发生

把"已闭合必须跳过"这条规则故意拿掉,用一个具体的图看错并怎么发生,比抽象论证更有说服力。构造: $0 \rightarrow 1$, $1 \rightarrow 0$ (环, SCC (0, 1)), 再 $2 \rightarrow 1$ (2 单向指向已成型的 (0, 1)), 2 自己无回路,应当是独立 SCC 2。访问次序 0, 1 先成环并收口,然后才访问 2。

动作	正确(挡已闭合)	错误(不挡,用 $\text{low}[1]$)
$\text{dfs}(0) \rightarrow \text{dfs}(1) \rightarrow 1 \rightarrow 0$ 回边	$\text{low}[1] = 0, \text{low}[0] = 0$	
$\text{dfs}(0)$ 收口	$\text{low}[0] == \text{dfn}[0] = 0$, 弹出 (0, 1), 0、1 标记为已闭合、出栈	
$\text{dfs}(2)$, 处理 $2 \rightarrow 1$	1 已闭合 → <code>processed</code> 挡掉, 跳过; $\text{low}[2]$ 保持 $\text{dfn}[2]$	无脑 $\text{low}[2] = \min(\text{low}[2], \text{low}[1] = 0) = 0$
$\text{dfs}(2)$ 判根	$\text{low}[2] == \text{dfn}[2] \rightarrow 2$ 独立成 SCC ✓	$\text{low}[2] = 0 \neq \text{dfn}[2] \rightarrow 2$ 不成根 ✗
后果	(0, 1) 和 2 正确分开	2 的分量归属错乱: 它指向的 (0, 1) 早已收口、根本回不去, 却因借用 $\text{low}[1] = 0$ 而误判 2 与更早的点同环

这就是第 4.2 节"错并"的**可见版本**: 2 只是单向指向一个已经收口的分量, 它和 (0, 1) 之间没有任何"回得来"的路, 本该独立。一旦用了已闭合点的 low , 2 就被错误地染上"能回到 $\text{dfn}=0$ "的属性, 从而 $\text{low} == \text{dfn}$ 判根失败, 分量归属彻底错乱。**"已闭合必须跳过"不是可有可无的优化, 而是正确性的硬约束——把这条约束落进代码的, 正是那个一物两用的栈(在不在栈 = 闭没闭合)。**

6 · 根判据与栈:如何一次性把一个分量捞干净

6.1 根判据 $\text{low}[u] == \text{dfn}[u]$

回溯到 u 、它所有出边处理完后,若 $\text{low}[u] == \text{dfn}[u]$,意味着" u 及其子树,最远也只能往回够到 u 自己,够不到任何比 u 更早的点"——那么 u 就是它所在 SCC 的**根**(分量内 dfn 最小的点),此刻应当收口。

这个判据对单点和多点分量统一成立,无需特判。一个孤立点(或没有任何回边的点)同样满足 $low[u] == dfn[u]$, 收口时只把自己弹出, 自成一个 size 为 1 的 SCC。还记得 1.1 里那句"零步可达自身"吗? 它在这里兑现了: 单点也是合法的等价类。凡是想给单点单独写 if 分支的, 都是没把判据想透。

6.2 用栈捞分量: 为什么是栈, 弹到哪为止

疑 ▶ $low[u] == dfn[u]$ 成立了, 我知道 u 是根了——可"这个分量到底包含哪些点"? 怎么一次把它们拿出来?

答 ▶ 用一个栈。规则: **每个点首次访问时入栈**; 当 $low[u] == dfn[u]$ 时, **从栈顶连续弹出**, 直到把 u 自己弹出为止——弹出的这一段(含 u) 恰好就是 u 这个 SCC 的全部成员。

疑 ▶ 凭什么"从栈顶弹到 u " 圈出来的, 不多不少正好是这个分量? 会不会把别的分量的点也弹进来?

答 ▶ 靠两条性质:

① **栈中元素的 dfn 必然递增**(从栈底到栈顶)——因为按访问顺序入栈, 先访问的 dfn 更小、更靠栈底。

② **一个 SCC 出栈后, 剩余的栈仍然保持 dfn 递增**——出栈只是从顶上削掉连续一段, 不破坏底下的递增性。

由这两条: 当 $low[u] == dfn[u]$ 时, 栈里所有 $dfn \geq dfn[u]$ 的点, 恰好是**栈顶的一段连续区间**, u 在这段的最底(它 dfn 最小)。而那些更早就已经收口的子分量, 早就被弹出栈了, 根本不在栈上, 不可能被误捞。所以"弹到 u 为止"(等价于"弹出所有 $dfn \geq dfn[u]$ 的点")圈出的, 正是 u 的整个分量, 一个不多一个不少。

修 ▶ 一个出栈条件的小细节: 用 " $dfn[x] \geq dfn[u]$ " 判断时, **等号不能漏**——漏了等号, 根 u 自己 ($dfn[x] == dfn[u]$) 就弹不出来, 分量会少一个根。等价的、更不易错的写法是直接"循环弹出, 直到弹出的那个点 $== u$ 为止"。两者圈出的是同一段, 因为栈 dfn 递增保证了 $\geq dfn[u]$ 的就是栈顶连续段。

栈一物两用——这是 Tarjan 的精妙所在: (1) 它用来**捞出**每个 SCC 的成员; (2) 它同时就是第 4 节那个"闭合标记"的实现——"**邻居 j 还在不在栈里**" 恰好等于 " **j 所属分量闭没闭合**"。一个点出栈, 等价于它所在分量被收口、闭合。于是第 4.4 节苦苦寻找的那个"必须额外记录的闭合状态", 答案就是"在不在栈里", 一分钱额外空间都不用多花(实现上常配一个 in-stack 布尔数组做 $O(1)$ 查询)。

7 · 正确性证明: 把前面所有判断串成一条不漏的链

到这里, 正确性其实已经在推导过程中被逐条建立了。我们把它收拢成一条形式化的链, 每一环都指回前面对应的小节:

1. **SCC 是相互可达的等价类**, V 被划成不相交的极大块。(§ 1.2–1.3, 由 $\sim$ 的等价关系性质)
2. **每个 SCC 在 DFS 树里有唯一的根**(分量内 dfn 最小的点), 且分量的所有点在栈上构成**连续**一段。(§ 6.2, 由"栈 dfn 递增 + 出栈不破坏递增")

3. **low[u] 的语义被严格固定为:**u 及其子树,经由"所属分量未闭合"的边,能够回到的最早 dfn。
(§ 3-§ 4,由"判断封顶"的需求逼出)
4. **更新规则正确:**只对"未闭合"的邻居取 min,已闭合的跳过;对回边用 dfn[j]、对已返回的树边孩子用 low[j]。因为已闭合的 j 与 u 连不成活环,其信息无效;而该并的环,其最早 dfn 必经由树边链汇聚到根。(§ 4.4、§ 5)
5. **根判据 low[u]==dfn[u] 充要:**u 够不到任何更早的点 $\iff$ u 是分量根 $\iff$ 此刻可安全收口。对单点统一成立。(§ 6.1)
6. **收口的完整性与互斥性:**low[u]==dfn[u] 时,栈中 dfn $\geq$ dfn[u] 的连续一段恰是 u 的整个 SCC,不多(子分量已先出栈)不少(等号含根)。每个点恰好被收口一次。(§ 6.2)

这六条合起来,即构成 Tarjan 算法的正确性证明:它**确实**把每个点划入**且仅划入**它真正所属的那个等价类,与第 1 节的纯定义完全吻合。每一步都有理由,无一处依赖"教科书说所以对"。

复杂度:每个点恰好入栈一次、出栈一次;每条边在其起点的循环里恰好被处理一次。故时间 $O(V+E)$,空间 $O(V)$ (dfn、low、栈、in-stack 标记)。我们达成了第 2 节立下的目标:把朴素的 $O(V^4)$ 压成线性,且是**一遍 DFS** 顺手完成。

7.1 把两条关键性质做成严格归纳

上面六条是证明的骨架。其中第 3 条(low 语义正确)和第 2/6 条(栈连续)最值得用**归纳法**补严,因为它们是其余结论的地基。

命题 A:当 dfs(u) 返回时,low[u] 恰等于 § 3.4 定义的那个 min。

对 u 子树的大小做归纳。基例:u 是叶子(无出边),循环不执行,low[u]=dfn[u];而 u 能到达的、满足定义的 w 只有它自己(零步),min dfn 即 dfn[u]。命题成立。

归纳步:设 u 的所有树边孩子 c 的子树都更小,由归纳假设,dfs(c) 返回时 low[c] 已等于其定义值。处理 u 的每条出边 (u→j):①若 j 是树边孩子,递归后 low[u]=min(low[u],low[j]) 把"经 u→j 进入子树"的所有可达早点纳入;②若 j 已访问且未闭合,按定义只允许"至多一条非树边",此步 min(low[u],low[j]/dfn[j]) 恰好纳入"u 一步跨到未闭合 j"这条路径;③若 j 已闭合,定义不允许经它(它不在"未闭合顶点"集合里),代码用 !processed 跳过。三种情况与定义逐项对应,故 u 返回时 low[u] 等于定义值。■

命题 B:栈中元素 dfn 始终严格递增(不变量 I3)。

对操作序列(push / pop)做归纳。初始栈空,平凡成立。push 只发生在"首次访问某点 u"时,而 u 的 dfn 是**当前最大**的发现序号(它刚被分配),故压到栈顶仍保持递增。pop 只从栈顶连续移除,不改变剩余元素的相对顺序,递增性保持。由归纳,任何时刻栈 dfn 递增。■

推论:收口段恰好是一个完整 SCC(第 6 条的严格化)。

由命题 B,low[u]==dfn[u] 时,栈中 dfn $\geq$ dfn[u] 者构成栈顶连续段 S,u 在其底。需证 S 恰为 u 的 SCC:

($\subseteq$)S 中任意 x 满足 dfn[x] $\geq$ dfn[u] 且仍在栈(未闭合),说明 x 在 u 的子树中、且能经未闭合路径回到 dfn $\leq$ dfn[u] 的点——结合 low[u]==dfn[u](u 回不到更早),可推出 x 与 u 互相可达,故 x 与 u 同 SCC。

($\supseteq$)u 的 SCC 中任意 y:y 与 u 互相可达,故 dfn[y] $\geq$ dfn[u](u 是该 SCC 中最早被访问者——否则更

早者会成根),且在 u 收口前 y 不可能已被收口(否则 u 早被并入 y 的分量),故 y 仍在栈、落在 S 中。两向夹逼, $S = u$ 的 SCC,不多不少。■

命题 A、B 与推论,把第 7 节开头六条骨架里最依赖"感觉"的几步,落成了可检验的归纳论证。至此,正确性不再有任何"显然"的缝隙。

8 · 从证明到代码的那道沟(本文核心二):逐个 bug 复盘

"懂算法"和"写对程序"之间,永远隔着一道沟。下面把这道沟里我**真实踩过**的**每一个坑**摆出来——它们无一例外,都是"某条推导没有正确落地到代码"的后果。这一节比任何"标准实现"都值钱,因为它告诉你**错会错在哪、为什么**。

8.1 坑一:low 的初值,以及"兜底"放错位置

疑► 我把 low 全初始化成 INT_MAX,然后在循环结束后写 `if(low[r]==INT_MAX)`
`low[r]=dfn[r]` 兜底——哪里不对?

修► 两个错。

(1) **逻辑错**:"没人更新过才设成 dfn[r]"——但正确语义是 low[r] 一开始就该等于 dfn[r](每个点至少能回到自己),再被邻居 min 拉小,不是"没人更新才兜底"。

(2) **位置错(致命)**:兜底在 for 循环**之后**。可回边在循环**之中**就要读祖先的 low——而此刻祖先的 low 还是 INT_MAX(它的 DFS 还没返回、还没轮到循环后的兜底)。于是回边 $u \rightarrow$ 祖先时, `min(low[u], INT_MAX)` 等于什么也没干,**回边信息被自己挡掉**。现象:所有环都散架成单点。

正解:进入 DFS 的**第一行**就写 `low[r] = dfn[r]`。这样任何时刻读任何点的 low 都是有效值,回边能正常 min。这也正呼应 §3 的语义:"我至少能回到我自己"是 low 的**底**,之后被一步步拉小——low 本就是动态收敛量,它的底必须从一开始就在。

8.2 坑二:processed 的下标坐标——节点 id 还是 dfn 序号?

疑► 我写 `processed[low[j]]` 来判断 j 闭没闭合,又在收口时写 `processed[dfn[x]]=true`。报了越界(segfault),为什么?

答► 表面是越界:low[j] 在 j 还没拿到有效值时是 INT_MAX, `processed[INT_MAX]` 直接出界。根治办法就是 8.1——初值修好后 low 永远落在 $[0, n)$ 。

更深一层是**坐标一致性**:processed 数组到底按"节点 id"索引,还是按"dfn 序号"索引?**两套坐标都行,但全程必须用同一套**。本文代码选择**按 dfn 序号索引**:读 `processed[low[j]]` (low 存的是 dfn 值)、写 `processed[dfn[x]]`,两处都用 dfn 坐标,自治。如果一处用节点 id、一处用 dfn,就会"标记了 A、查询了 B",逻辑全乱。**dfn 与节点一一对应,但"一一对应"不等于"可以混用下标"**——它们是两套编号。

8.3 坑三:判据查错了主语

修► 一个隐蔽的语义错:判据是"**j 对应的点**闭没闭合",不是"**low[j]** 这个数值对应的点闭没闭合"。在按 dfn 索引、且 low/dfn 坐标统一的写法下,processed[low[j]] 恰好查的就是"**j 能够回到**的那个代表点的闭合态",在本算法的不变量下与"**j 所属分量**闭合态"一致,所以成立。但如果你脑子里把主语搞成了别的点(比如某个无关的横叉目标),就会写出查错对象的代码。**写每一个数组下标前,先问一句:我现在要的是"哪个点"的状态?**

8.4 坑四:收口循环忘了出栈

修► 收口时 while 循环里读了 st.top()、标记了 processed、push 进了 scc——却忘了 st.pop()。结果栈永不缩减,要么死循环、要么把同一批点反复收。纯属手滑,但极常见。**凡是"从栈顶取了东西并消费掉"的循环,当场就要配一个 pop。**

8.5 坑五:出栈判据漏等号 / 用错量

修► § 6.2 已说过:用 $\text{dfn}[x] \geq \text{dfn}[r]$ 时**等号不能漏**,否则根弹不出。更稳的写法是"弹到 $x==r$ 为止"。另外别把出栈判据写成比较 low(应该比 dfn)——low 是会变的,拿它当弹栈边界会圈错范围。

8.6 一条通用的自检纪律

把"懂"翻译成"对"的唯一可靠办法:写完后,手动拿一个最小触发用例 trace 一遍。对 SCC,最小用例就是 $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$ 这个三元环。逐行走:当 DFS 走到节点 2、它指回 0 时,问自己——此刻 low[0] 是多少?它是有效值还是 INT_MAX?processed[low[0]] 查的是哪个点?该不该更新?——这一遍 trace 能在 30 秒内把上面五个坑里的大多数当场照出来。**真正的 bug 往往不在"想不到",而在"想到了却默认自己会写对、跳过了验证"。**

8.7 边界与陷阱清单

除了上面五个逻辑坑,还有一批"输入形态"上的边界,值得在写完后逐条对照——它们不一定让你 WA,但漏了会在特定数据上翻车:

- **重边(平行边):**u 到 v 有多条边。Tarjan 完全不受影响——重复处理同一条 ($u \rightarrow v$) 只是多做一次 min/判断,结果幂等。无需去重。
- **自环 $u \rightarrow u$:**处理到自己时 u 已访问、在栈中, $\min(\text{low}[u], \text{low}[u])$ 无副作用;u 仍按 $\text{low} == \text{dfn}$ 单独判根。自环不会让 u 变成多点分量,除非另有回路。代码无需特判自环。
- **不连通 / 多个 DFS 树根:**图可能有多个互不可达的部分。所以主循环必须 `for i in 0..n-1: if !vis[i] dfs(i)` ——**对每个未访问点都起一次 DFS**,否则会漏掉孤立的子图。这是最常见的"只跑了一个连通块"的错。
- **孤立点(无任何出边):**dfs 进去,for 循环不执行,直接 $\text{low} == \text{dfn}$ 成根,弹出自己,size-1 分量。由单点判据统一覆盖,见 § 6.1。
- **空图($n=0$):**主循环不执行,返回空。无需特判。

· **cnt 计数器的传递:**本文用引用 `int& cnt` 跨递归共享同一个全局发现序计数器。若误写成值传递, 每棵 DFS 树的 `dfn` 会从 0 重启, 不同树的 `dfn` 撞号, 栈 `dfn` 递增性被破坏, 收口全错。这是个隐蔽但致命的语言层错误。

一句话验收:重边/自环天然安全; 不连通务必主循环遍历所有起点; 孤立点/空图由统一判据兜住; `cnt` 必须全局共享(引用传递)。把这张清单和 § 8.1–8.6 的五个逻辑坑一起过一遍, 你的实现就既"对"又"健壮"了。

下面是完整可运行实现(C++17), 已通过七个用例: 单环、无环链、环挂尾、双分量、共享点合并成一个大 SCC、自环+孤立点、较大混合。每个关键落地点都在前文有对应推导: 进门即设 `low[r]=dfn[r]` (§ 8.1)、`processed` 按 `dfn` 索引且仅对未闭合邻居取 `min`(§ 4.4/§ 8.2)、收口按 `dfn[x] ≥ dfn[r]` 连续弹出并 `pop`(§ 6.2/§ 8.4)。

```
#include <climits>
#include <vector>
#include <stack>
#include <functional>
#include <algorithm>
using namespace std;

vector<vector<int>> tarjanSCC(int n, const vector<vector<int>>& g) {
    vector<int> dfn(n, -1), low(n, INT_MAX);
    vector<bool> vis(n, false), processed(n, false);
    stack<int> st;
    vector<vector<int>> ret;

    function<void(int,int&)> dfs = [&](int r, int& cnt) {
        dfn[r] = cnt++;
        low[r] = dfn[r];          // KEY: set on entry; every node can reach itself
        st.push(r);
        for (int idx = 0; idx < (int)g[r].size(); ++idx) {
            int j = g[r][idx];
            if (!vis[j]) {        // tree edge: recurse first
                vis[j] = true;
                dfs(j, cnt);
            }
            if (!processed[low[j]]) // use low[j] only if j's component is NOT closed
                low[r] = min(low[r], low[j]);
        }
        if (low[r] == dfn[r]) {    // r is the root of an SCC -> close it
            vector<int> scc;
            while (!st.empty()) {
                int x = st.top();
                if (dfn[x] >= dfn[r]) {
                    processed[dfn[x]] = true; // mark closed (indexed by dfn)
                    scc.push_back(x);
                    st.pop();
                } else break;
            }
            ret.push_back(scc);
        }
    };

    int cnt = 0;
    for (int i = 0; i < n; ++i)
        if (!vis[i]) { vis[i] = true; dfs(i, cnt); }
    return ret;
}
```

七个用例覆盖了所有边界:纯环验证合并、无环链验证"每点自成一分量"、环挂尾验证分量分界、双分量与共享点验证"合并 vs 不合并"、自环+孤立点验证单点判据的统一性、较大混合做综合回归。全部 PASS,exit 0。

9.1 亲手 trace 一遍:0→1→2→0 的完整执行状态表

口说无凭,把第 8.6 节的自检纪律落到实处——逐行走一遍最小环 0→1→2→0 (邻接表 $g = [[1],[2],[0]]$),记录每一步的 dfn、low、栈、以及关键判断。这张表既是验证,也是把前面所有推导"看见"的最好方式。

步	动作	dfn	low	栈(底→顶)	说明
1	进 dfs(0)	0:0	0:0	[0]	进门即 $low[0]=dfn[0]=0$
2	0→1,进 dfs(1)	0:0 1:1	0:0 1:1	[0,1]	1 未访问,树边递归
3	1→2,进 dfs(2)	0:0 1:1 2:2	0:0 1:1 2:2	[0,1,2]	2 未访问,树边递归
4	2→0,回边	不变	$2:0(\leftarrow \min(2, low[0]=0))$	[0,1,2]	0 已访问且 未闭合(在栈) ,可用; $low[2]=\min(2,0)=0$
5	dfs(2) 收尾	不变	2:0	[0,1,2]	$low[2]=0 \neq dfn[2]=2$,2 不是根,不收口
6	回到 1,用 low[2]	不变	$1:0(\leftarrow \min(1, low[2]=0))$	[0,1,2]	树边孩子返回,用 $low[2]; low[1]=0$
7	dfs(1) 收尾	不变	1:0	[0,1,2]	$low[1]=0 \neq dfn[1]=1$,不收口
8	回到 0,用 low[1]	不变	$0:0(\leftarrow \min(0, low[1]=0))$	[0,1,2]	$low[0]=0$
9	dfs(0) 收尾,判根	不变	0:0	[0,1,2]	$low[0]==dfn[0]==0$,0 是根!收口
10	弹栈 (dfn≥0)	—	—	[]	弹出 2、1、0(全部 $dfn \geq 0$),组成 SCC (0, 1, 2)

看第 4 步——这正是第 8.1 节那个坑的现场:此刻 $low[0]$ 必须已经是有效值 0(进门就设),回边才能把 $low[2]$ 拉到 0。如果 $low[0]$ 还是 INT_MAX (兜底拖到循环后),第 4 步的 $\min$ 会失效, $low[2]$ 停在 2,接着 $low[1]$ 、 $low[0]$ 都拉不下来,第 9 步每个点都会满足 $low==dfn$ 、各自成根,环散成三个单点——和我们当初的 bug 现象完全一致。**一张表,把"为什么初值必须进门设"彻底坐实。**

再看第 4 步的"可用"判断:0 此刻在栈里(未闭合),所以 $!processed[low[0]]$ 为真,允许更新。设想一个对照:若 0 所在分量已经收口出栈,这条回边就会被 $processed$ 挡掉——这正是第 4.4 节判据在代码里的样子。

10 · 它从哪来:图灵奖与 1972 那篇论文

强连通分量的线性算法出自 Robert Tarjan 1972 年的论文《Depth-first search and linear graph algorithms》。这篇论文真正的贡献,不只是"求 SCC",而是**开创了一种范式:用一遍 DFS,加上一个巧妙设计的辅助量**(本文的 low,即 low-link),把一大类看似需要多遍才能解决的图问题,统统压成线性时间。线性求 SCC 只是这个范式最著名的应用之一,同篇还含双连通分量等。

Tarjan 与 John Hopcroft 因"在算法与数据结构的设计与分析上的根本性成就",共同获得 **1986 年图灵奖**。SCC / low-link 正是获奖工作的代表性成员。Tarjan 的招牌手法一以贯之:**一遍 DFS + 一个聪明的附加量,把多遍压成线性**——支配树(dominator tree)、离线 LCA、以及并查集那个近乎常数的摊还复杂度分析,都是同一思想的不同化身。理解了本文这一个 low,你其实已经摸到了他整套工作的脉门。

11 · 余论:欧拉七桥、建模 > 求解、第一性原理与手稿

写到这里,值得退一步谈谈方法论——因为它解释了"为什么本文要这么费劲地正着推",也解释了为什么经典离散算法虽然结论简洁,却配得上被反复咀嚼。

11.1 欧拉七桥:建模 > 求解

常见误解是"经典离散算法很深奥"。但真正稀缺的从来不是最后那个简洁结论,而是**敢把无关的东西全扔掉、只留结构**的那一下。欧拉对七桥问题本人觉得近乎 trivial——他在给但泽市长的信里说,解法"只基于常识,不依赖任何数学原理"。可是"trivial"是**后验**的:它锚定在"图"这个对象已经成形之后。而**把一张现实地图抽象成"点 + 边"**这一步——丢掉桥的形状、长度、地理,只保留连接关系——人类要再等近 150 年才普遍做到。难的不是"度数奇偶决定能否一笔画"这个判据(抽象到位后它比较自然地掉出来),而是**发明那个坐标系本身**。

书信考据顺带印证了这点:Ehler 在 1736 年 3 月 9 日正式请欧拉给解;欧拉 3 月 13 日致 Marinoni 时已宣布"找到一个不止适用此例、而适用所有同类问题的证明";4 月 3 日才正式回 Ehler。而他其实早在 1735 年 8 月就向圣彼得堡科学院报告过此题——也就是说"解"早有腹稿,那几周的间隔是回信节奏,不是解题耗时。**建模 > 求解:抽象对了,解往往就不远了**。本文反复强调"先把问题钉死(§ 1)、让笨办法替你指方向(§ 2)",正是这条原则在算法层面的实践。

11.2 第一性原理 = 一座借来的公理库

你用来当"公理"直接调用的那套第一性原理,本身并非公理,而是被前人证明过、然后压缩进你直觉的**定理**——你只是不再为它们付推导的代价。这是第一性原理思考的强项(快),也是它的风险:你信任的那一层若藏着暗坑,整座推理会跟着歪,而你浑然不觉,因为你早已不去检查它。所以真正强的做法,是**周期性地往下捅一层**,问自己:"我当公理用的这个东西,到底是定理,还是只是习惯?"

本文做的,正是往下捅了一层。大多数人把 low 当公理背下来;这里我们把它从"判断分量封顶"的需求重新逼了出来。一个公理被拿掉、还能从更深的层重建出来,说明这份知识不是悬空的卡片,而是**连通的网络**——这,才是"真懂"与"背过"之间那条真正的分界线。被面试官追到底层时不慌,靠的就是这张网,而不是那张卡。

11.3 手稿即源码

最后一点,关于本文的体例本身。一篇正式论文,是把数月的混乱思考**压缩**成几页线性叙述——读者看到的是"定理 → 证明 → 结论"的干净链条,而作者真实走过的路(试了 A 不行、绕到 B、发现和 C 有关、推翻重来)只活在**手稿**里。清稿是有损压缩,手稿才是源码。本文刻意保留了大量"疑/答/修"的拉扯,就是要把一次真实的"手稿"清成"带注释的源码",而不是再生产一份干净却没有来路的清稿。它的价值不在最终的整洁,而在**保留了发现的曲折**——因为那条曲折的路,才是把"知道"变成"拥有"的唯一通道。

问题 · 逼出 low · 三类边的拉扯 · dfn 与 low 之争 · 根判据与栈 · 证明 · 从证明到代码的沟 · 思想史 —— 一次完整的、正着推的过程。